Walrus Operator in Python

Assignment expressions with := (Python 3.8+): assign and use in one step

What it does

`name := expression` **assigns** the value and also **returns** it, so you can assign inside an `if`, `while` or comprehension. Named for the eyes and tusks of `:=`.

```
print(n := 10)          # assigns and prints
print(n)
```

OUTPUT

```
10
10
```

Syntax rules

Form	Valid?
`x := 5` as a statement	SyntaxError
`(x := 5)` parenthesised	OK
`if x := f(): while x := f():`	OK, no extra parentheses
`d[k] := v`, `obj.a := v`	SyntaxError (names only)
`(a, b) := (1, 2)`	SyntaxError

Avoid calling twice

```
data = [3, 9, 4]
if (n := len(data)) > 2:
    print(f"{n} items is a lot")
```

OUTPUT

```
3 items is a lot
```

Input loop without repeating yourself

Before: read once before the loop, then again at the bottom. After: one line.

```
lines = iter(["a", "b", ""])
while (line := next(lines)) != "":
    print("got", line)
```

OUTPUT

```
got a
got b
```

Read chunks

```
import io
f = io.StringIO("abcdefg")
while chunk := f.read(3):
    print(chunk)
```

OUTPUT

```
abc
def
g
```

TIP An empty result is falsy, so the loop ends at end-of-file: no `!= ""` needed.

Filter and keep the computed value

Compute once per item and reuse it in the output.

```
words = ["hi", "hello", "yo", "world"]
[(w, n) for w in words if (n := len(w)) > 2]
# → [('hello', 5), ('world', 5)]
```

Regex match then use it

```
import re
if m := re.search(r"(\d+)-(\d+)", "tel 555-1234"):
    print(m.group(1), m.group(2))
```

OUTPUT

```
555 1234
```

Handy with any() and all()

Capture which item satisfied the test.

```
nums = [1, 4, 9, 12]
if any((big := n) > 8 for n in nums):
    print("first big:", big)
```

OUTPUT

```
first big: 9
```

Gotchas

```
# without parentheses x gets the bool
if x := len("ab") > 1:
    print(x)                # → True
if (x := len("ab")) > 1:    # parenthesised
    print(x)                # → 2
```

WATCH OUT `:=` has the lowest precedence: without parentheses it captures the whole comparison. Overusing it hurts readability; keep it to simple cases.

REMEMBER

- `(name := expr)` assigns **and** returns the value: use it to avoid computing something twice.
- Parenthesise unless it is the whole condition: `if (n := f()) > 5`.
- It only binds a plain name: not `d[k]`, not `obj.attr`, not tuples.
- A name assigned in a comprehension leaks into the enclosing scope.
- Great for `while chunk := f.read(n)` and `if m := re.search(...)`.
- If a plain assignment on its own line reads better, use that.